1 DOCUMENTATION

Nom	PASCAL BENZONANA	Hussein Sofian - Cunha Faria Joël-Theytaz Samuel
Contact	pascal.benzonana@eduvaud.ch	sofian.hussein@eduvaud.ch joel.cunhafaria@eduvaud.ch samuel.theytaz@eduvaud.ch

Contexte	Développement sécurisé d'une application Web conforme aux recommandations OWASP
Date	Du vendredi 6 février au 20 mars 2026
Objet	Cours ICT-183 - Documentation

Ce projet est réalisé dans le cadre du module ICT-183. Il consiste à développer une application web de location de voitures en ligne, tout en mettant l'accent sur la sécurité applicative.

Nous vous présentons par le biais de ce document, les choix de conception du projet ainsi que les mesures de sécurité mises en place.

L'objectif principal est d'identifier et de corriger les vulnérabilités du Top 10 OWASP 2021 (celle réalisable dans notre cas), afin de protéger l'application contre les menaces courantes comme les injections, les problèmes d'authentification ou l'exposition de données sensibles.

L'application est développée avec Express (backend), Vue.js (frontend), Tailwind CSS (style) et MySQL (base de données).

2 SOMMAIRE

1 DOCUMENTATION 1
2 SOMMAIRE 2
3 INTRODUCTION 4
3.1 Contexte du module ICT-183 4
3.2 Objectif du projet 4
4 Les failles concernées 5
4.1 A01 Broken Access Control 5
4.2 A02 Cryptographic Failures 5
4.3 A03 Injection 5
4.4 A04 Insecure Design 6
4.5 A05 Security Misconfiguration 6
4.6 A07 Identification and Authentication Failures 6
5 PRESENTATION DU PROJET 7
5.1 FONCTIONNEMENT GENERAL 7
5.2 TECHNOLOGIES UTILISEES 7
5.3 Backend 7
5.4 Frontend 7
5.5 Base de données 8
5.6 Interface utilisateur 8
6 Architecture du projet 9

7 Mesures de sécurité mises en place 10
7.1 Authentification et identification forte 10
7.2 Protection contre les injections SQL 11
7.3 Protection des données et gestion des secrets 11
7.4 Contrôle d'accès 12
7.5 Sécurité réseau et configuration du serveur 12
7.6 Gestion centralisée des erreurs 13
7.7 Protection contre les attaques XSS 13
8 Améliorations possibles 14
9 Conclusion 14

3 INTRODUCTION

3.1 Contexte du module ICT-183

L'objectif est de comprendre les problèmes de sécurité qui peuvent apparaître dans une application web et d'apprendre comment les éviter. Nous découvrons notamment comment certaines attaques fonctionnent et quelles solutions peuvent être mises en place pour protéger une application.

Ce projet nous permet donc de mettre en pratique ces notions en développant une application réelle en intégrant des mesures de sécurité dès le début du développement.

3.2 Objectif du projet

L'objectif de ce projet est de développer une petite application web de location de voitures en ligne. Les utilisateurs doivent pouvoir créer un compte, se connecter et réserver une voiture directement depuis l'application.

En plus de la partie fonctionnelle, le projet met aussi l'accent sur la sécurité de l'application. Nous devons donc appliquer les bonnes pratiques vues pendant le module afin de protéger l'application contre les failles de sécurité les plus courantes.

Ce projet permet ainsi de travailler plusieurs aspects importants du développement :

La création d'une application web complète mais minimalisé

L'utilisation de technologies modernes pour le frontend et le backend

La gestion d'une base de données

L'intégration de mesures de sécurité dans l'application en fonction de OWASP

Le but est d'obtenir une application qui fonctionne correctement tout en étant sécurisée.

4 *Les failles concernées*

Dans ce projet, la sécurité l'élément essentiel. Nous nous appuyons sur les recommandations de l'OWASP sur le Top 10 (2021) qui liste les dix failles de sécurité actuellement les plus fréquentes dans les applications web.

Les failles que nous allons couvrir :

4.1 A01 Broken Access Control

Cette faille apparaît lorsqu'un utilisateur peut accéder à des ressources ou effectuer des actions auxquelles il ne devrait pas avoir accès. Par exemple, un utilisateur normal pourrait accéder à des fonctions réservées à un administrateur. Cela se produit lorsque les vérifications des permissions sont mal implémentées.

4.2 A02 Cryptographic Failures

Cette vulnérabilité concerne une mauvaise utilisation de la cryptographie. Par exemple, stocker des mots de passe en clair ou utiliser des méthodes de chiffrement faibles. Cela peut permettre à un attaquant d'accéder à des données sensibles.

4.3 A03 Injection

Une attaque par injection se produit lorsque des données envoyées par un utilisateur sont interprétées comme du code par l'application. Par exemple, une injection SQL peut permettre à un attaquant de manipuler une base de données. Cela arrive généralement lorsque les entrées utilisateur ne sont pas correctement validées.

4.4 A04 Insecure Design

Cette faille est liée à des problèmes dans la conception même de l'application. Si la sécurité n'est pas prise en compte dès le début du projet, certaines protections peuvent manquer. Cela peut créer des vulnérabilités difficiles à corriger par la suite. Dans le cadre de ce projet, cette problématique est traitée au niveau de la documentation : les choix d'architecture, la séparation des rôles, la gestion des tokens et la structuration des routes ont été pensés en tenant compte des bonnes pratiques de sécurité dès la phase de conception.

4.5 A05 Security Misconfiguration

Cette vulnérabilité est liée à une mauvaise configuration du système, du serveur ou de l'application. Par exemple, laisser des pages d'administration accessibles publiquement ou utiliser des paramètres par défaut non sécurisés. Ces erreurs peuvent faciliter l'exploitation de l'application par un attaquant.

4.6 A07 Identification and Authentication Failures

Cette vulnérabilité concerne les problèmes liés à l'authentification des utilisateurs. Par exemple, des mots de passe faibles, une mauvaise gestion des sessions ou l'absence de protection contre les tentatives de connexion répétées. Cela peut permettre à un attaquant d'accéder au compte d'un utilisateur.

5 PRESENTATION DU PROJET

5.1 FONCTIONNEMENT GENERAL

L'application permet aux utilisateurs de louer une voiture directement en ligne. Le fonctionnement est simple et se base sur trois étapes principales : la création d'un compte, la connexion à l'application et la réservation d'un véhicule.

Tout d'abord, l'utilisateur doit créer un compte en fournissant les informations nécessaires. Une fois le compte créé, il peut se connecter à l'application grâce à ses identifiants.

Après la connexion, l'utilisateur peut consulter les voitures disponibles et effectuer une réservation en sélectionnant le véhicule souhaité. La réservation est ensuite enregistrée dans la base de données de l'application.

5.2 TECHNOLOGIES UTILISEES

Le projet repose sur plusieurs technologies modernes permettant de développer l'application web.

5.3 Backend

Le backend de l'application est développé avec Express. Nous avons choisi ce framework pour sa manière de créer rapidement des API web performantes en Node.js et facilite la gestion des routes, des requêtes et de la sécurité.

5.4 Frontend

L'interface utilisateur est développée avec Vue.js, un framework JavaScript qui permet de créer des interfaces web dynamiques et interactives. Vue.js permet nativement de protéger les sites web contre l'injection XSS.

5.5 Base de données

Les données de l'application sont stockées dans une base de données MySQL. Elle permet de gérer les informations des Voitures, Locations, Clients, Paiements, Refresh token et des Agences.

5.6 Interface utilisateur

Le design de l'application est réalisé avec Tailwind CSS. Ce framework nous facilite la création d'une interface moderne et responsive grâce à des classes CSS déjà prêtes à l'emploi.

6 Architecture du projet

Nous avons choisi une architecture en couches pour structurer le backend de manière claire et maintenable. Chaque couche a un rôle précis : les routes dirigent les requêtes, les controllers valident les données entrantes, les services contiennent la logique métier et les repositories gèrent les accès à la base de données. Cette séparation permet de modifier une partie du code sans impacter les autres, et rend le projet plus facile à reprendre et à faire évoluer. Cela facilite aussi la gestion des erreurs.

7 Mesures de sécurité mises en place

7.1 Authentification et identification forte

Les mots de passe sont hachés avec bcrypt, un algorithme intentionnellement lent qui augmente le coût d'une attaque par force brute. Un facteur de coût de 12 est appliqué lors de l'inscription, ce qui produit automatiquement un sel aléatoire unique par mot de passe. Aucun mot de passe n'est stocké ou transmis en clair.

L'authentification repose sur deux tokens JWT distincts. L'access token, signé avec un secret dédié (JWT_ACCESS), a une durée de vie courte de 15 minutes. Le refresh token, signé avec un secret différent (JWT_REFRESH), est valable 7 jours et permet de renouveler l'access token sans que l'utilisateur ait à se reconnecter. Cette séparation limite l'impact d'un access token compromis.

Le refresh token est persisté en base de données dans la table refresh_tokens avec sa date d'expiration. Chaque requête de renouvellement vérifie à la fois la signature cryptographique du token et sa présence en base. Lors de la déconnexion, le token est supprimé de la base, ce qui permet la révocation immédiate de la session. Un job automatique s'exécute toutes les 24 heures pour purger les tokens expirés.

Le refresh token est transmis exclusivement via un cookie HttpOnly, ce qui le rend inaccessible au JavaScript côté client et protège contre les attaques XSS visant à voler les tokens de session. L'access token, lui, est renvoyé dans le corps de la réponse JSON et stocké uniquement en mémoire (Pinia store), sans persistance locale.

Les erreurs d'authentification retournent systématiquement le message générique « invalid credentials » que l'échec provienne d'un email inconnu ou d'un mauvais mot de passe, évitant ainsi toute fuite d'information sur l'existence d'un compte.

Le format des adresses e-mail est validé côté serveur via une expression régulière avant tout traitement, tant à la connexion qu'à l'inscription. Les champs obligatoires sont vérifiés systématiquement et une tentative d'inscription avec un e-mail déjà utilisé est rejetée avec un code 409.

7.2 Protection contre les injections SQL

L'ensemble des requêtes vers la base de données utilise exclusivement des requêtes préparées avec paramètres liés (placeholders « ? ») via le driver mysql2. Aucune concaténation de chaîne n'est utilisée pour construire des requêtes SQL. Cette approche garantit que les données saisies par un utilisateur sont toujours traitées comme des valeurs et jamais comme du code SQL, rendant les attaques par injection SQL inopérantes.

Ce principe est appliqué de façon homogène dans tous les repositories : authentification (recherche par email, gestion des refresh tokens), gestion des voitures, des locations et des paiements. Les paramètres provenant du corps de la requête HTTP ou du token JWT sont systématiquement passés en paramètres liés, jamais interpétés directement.

7.3 Protection des données et gestion des secrets

Tous les secrets de l'application (identifiants de base de données, clés de signature JWT, origine CORS autorisée) sont externalisés dans des variables d'environnement et ne figurent jamais en dur dans le code source. Le fichier .env est exclu du contrôle de version via .gitignore. Un fichier .env.example est fourni pour documenter les variables nécessaires sans exposer les valeurs réelles.

Les deux clés JWT (access et refresh) sont distinctes, ce qui signifie qu'une compromission de l'une ne permet pas de forger l'autre. Les tokens JWT embarquent uniquement les informations strictement nécessaires : l'identifiant client et le rôle. Aucune donnée personnelle sensible n'est incluse dans le payload des tokens.

La connexion à la base de données s'effectue via un pool mysql2 dont tous les paramètres (hôte, port, utilisateur, mot de passe, nom de base) sont injectés exclusivement depuis les variables d'environnement.

7.4 Contrôle d'accès

Chaque route protégée passe obligatoirement par le middleware authMiddleware, qui vérifie la présence et la validité cryptographique de l'access token JWT fourni dans le header Authorization. Si le token est absent, mal formé ou expiré, la requête est rejetée avec un code 401 avant d'atteindre le controller.

Les routes d'administration sont protégées par un deuxième middleware, requireRole, qui vérifie que le rôle présent dans le payload du token correspond au rôle requis. Un utilisateur authentifié sans le rôle « admin » reçoit un code 403. Les deux middlewares sont chapitrés en série, garantissant qu'un accès administrateur nécessite à la fois une session valide et le bon rôle.

Côté frontend, le router Vue.js applique également des guards de navigation. Les routes marquées requiresAuth redirigent vers la page de connexion si aucun token valide n'est obtenu. La route d'administration vérifie en plus que le rôle stocké dans le store Pinia est bien « admin », avec une redirection vers l'accueil dans le cas contraire. Ces vérifications côté client viennent en complément des contrôles serveur, sans les remplacer.

Pour les opérations sur les locations, l'identifiant client est extrait directement du payload du token JWT vérifié (req.user.id) et non depuis le corps de la requête. Cela empêche un utilisateur authentifié de soumettre un identifiant arbitraire pour accéder aux données d'un autre client.

7.5 Sécurité réseau et configuration du serveur

Le middleware Helmet est activé sur l'ensemble du serveur. Il positionne automatiquement plusieurs en-têtes HTTP de sécurité : Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security et d'autres, réduisant la surface d'attaque liée aux en-têtes par défaut d'Express.

Un limiteur de débit (rate limiting) est configuré via express-rate-limit : au maximum 100 requêtes par adresse IP sur une fenêtre glissante de 15 minutes. Ce mécanisme limite l'impact des attaques par force brute sur les endpoints d'authentification ainsi que les tentatives d'énumération ou de déni de service applicatif.

La politique CORS est restreinte à une seule origine autorisée, définie par la variable d'environnement ALLOWED_ORIGIN. Seules les requêtes provenant de cette origine peuvent interagir avec l'API, ce qui bloque les appels cross-origin non légitimes. L'option credentials: true est activée pour permettre la transmission du cookie de refresh token.

7.6 Gestion centralisée des erreurs

Toutes les erreurs applicatives sont interceptées par un middleware de gestion d'erreurs centralisé (errorMiddleware). Les erreurs non gérées retournent un code 500 avec un message générique, sans exposer de trace de pile, de message d'exception interne ou d'information technique. Chaque service métier crée des objets erreur avec un code HTTP explicite, transmis via le mécanisme next(error) d'Express pour éviter la fuite d'informations sensibles dans les réponses.

7.7 Protection contre les attaques XSS

Vue.js échappe nativement toutes les données affichées dans les templates grâce à sa liaison de données réactive. Les valeurs injectées via les expressions « {{ }} » ou les directives :bind sont automatiquement encodées en HTML, empêchant l'exécution de scripts injectés. Cela protège l'affichage des données issues de la base (noms, descriptions de voitures, informations client) contre les attaques XSS réfléchies et stockées.

Le stockage du refresh token dans un cookie HttpOnly renforce cette protection : même si du code JavaScript malveillant venait à s'exécuter, il ne pourrait pas accéder au cookie de session ni usurper l'identité de l'utilisateur.

8 Améliorations possibles

Malheureusement par manque de temps, nous n'avons pas intégré un système de logs qui auraient été très intéressant en termes de sécurité pour pouvoir analyser ce qu'il se passe sur le serveur. Il aurait aussi été intéressant d'ajouter une authentification à double facteurs.

9 Conclusion

L'application couvre les principaux points du Top 10 OWASP 2021 : requêtes préparées contre les injections SQL, double token JWT avec révocation, contrôle d'accès par middlewares, Helmet, rate limiting et CORS strict. Les secrets sont externalisés et les erreurs gérées sans fuite d'information. La sécurité a été intégrée dès la conception, ce qui constitue l'essentiel de l'exercice.

10 Source

- Corrections orthographiques : https://claude.ai/
- Prompt : relève-moi toutes les fautes d'orthographe du document que je te mets en annexe.
- Liste des failles OWASP : https://owasp.org/Top10/2021/fr/